CS 214: Data Structures

General Lists

Slide contents follow

Kruse and Leung "Data Structures & Program Design in C"

Prepared by:

Waleed A. Yousef, Ph.D.

© Waleed A. Yousef 2008

Motivation: Why `Lists`?

- **In a general list:**
  - new values are added in position determined by the user.
  - Element is removed from a position determined by the user.

- **Important notice:**
  - if we keep adding and removing from the first position (the head of the list) the general list will behave as a stack.

  - If we keep adding from one end and removing from another end the list will behave as a queue.

- **Application:**
  - In queues, sometimes we need a priority for some elements. We may need to put an emergency call prior to others.

© Waleed A. Yousef 2008

Motivation: Why `Lists`?



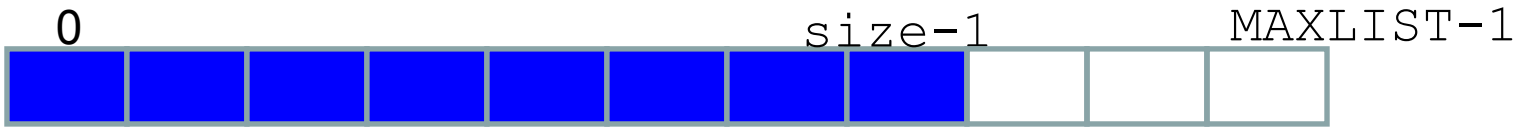
- This is a list (just a logical view, no implementation yet) with number of entries equals to `size`
- We can add a new element in position `0 ≤ p ≤ size.`
- However, we delete from `0 ≤ p ≤ size-1.`
- Now, let us be rigorous and define lists.

© Waleed A. Yousef 2008

Definition: A ***general list*** of elements of type T is a finite sequence of elements of T together with the following operations:

1. Create the list, leaving it empty.
2. Determine whether the list is empty or not
3. Determine whether the list is full or not
4. Find the size of the list.
5. Insert a new entry in the position $0 \le p \le size$.
6. Delete an entry from the position $0 \le p \le size-1$
7. Traverse the list, visiting each entry
8. Clear the list to make it empty

© Waleed A. Yousef 2008




```
void InsertList(int p, ListEntry e, List *pl);
```

Precondition:

1- The list pl has been created.

2- not full

3- 0 ≤p≤ size.

Postcondition:

1- e has been inserted at position p

2- all elements at old positions p, p+1, …, size-1 are incremented by 1.

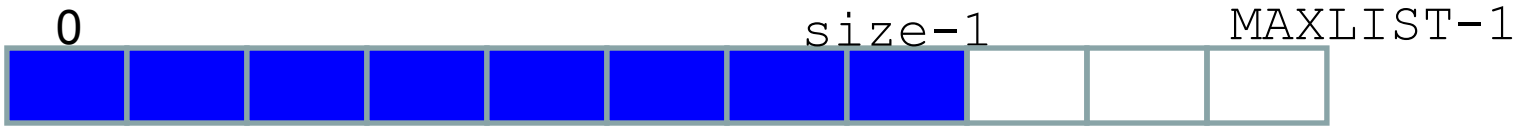
3- size increases by 1.

```
void DeleteList(int p, ListEntry *pe, List *pl);
```

Precondition: The list pl has been created, not empty, and 0 ≤ p ≤ size-1.

Postcondition: e has been retrieved from position p, and all elements at old positions p+1, …, size-1 are decremented by 1. size decreases by 1.

© Waleed A. Yousef 2008




```
void RetrieveList(int p, ListEntry *pe, List *pl);
```

same precondition as DeleteList. And the list is unchanged

```
void ReplaceList(int p, ListEntry e, List *pl);
```

same precondition. And the element is replaced

Other functions have similar pre- and post-conditions to the Queue and Stack.

Now let us start the Contiguous (array-based) Implementation

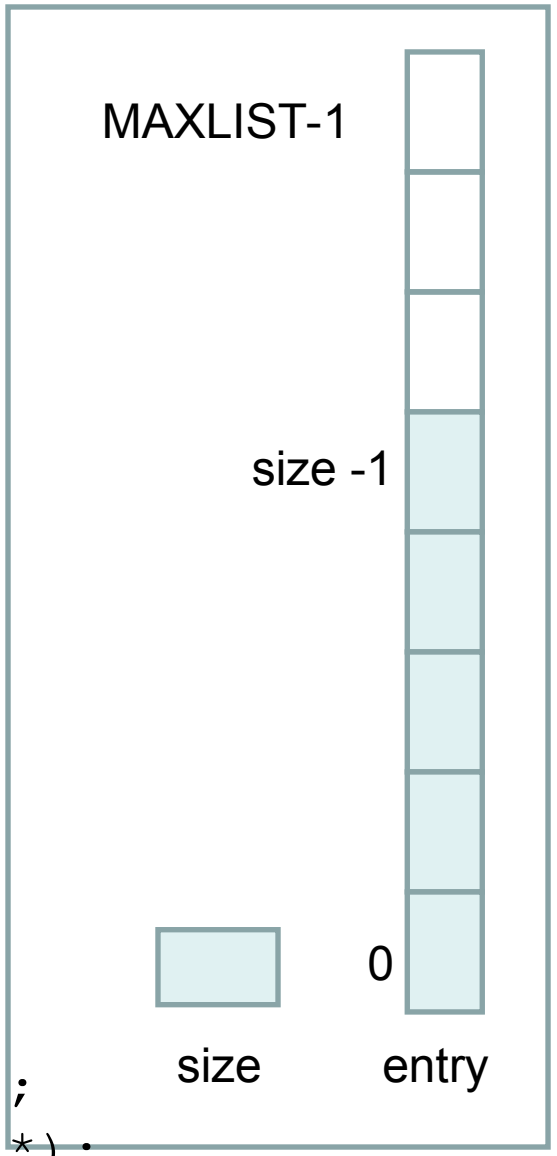
© Waleed A. Yousef 2008

```
/*List.h*/

#include "Global.h"

typedef struct list{
      ListEntry entry[MAXLIST];
      int size;
}List;

void  CreateList  (List *);
int   ListEmpty   (List *);
int   ListFull    (List *);
int   ListSize    (List *);
void  DestroyList (List *);
void  InsertList  (int, ListEntry, List *);
void  DeleteList  (int, ListEntry *, List *);
void  TraverseList(List *, void (*Visit)(ListEntry));
void  RetrieveList(int , ListEntry *, List *);
void  ReplaceList (int, ListEntry, List *);
```



© Waleed A. Yousef 2008

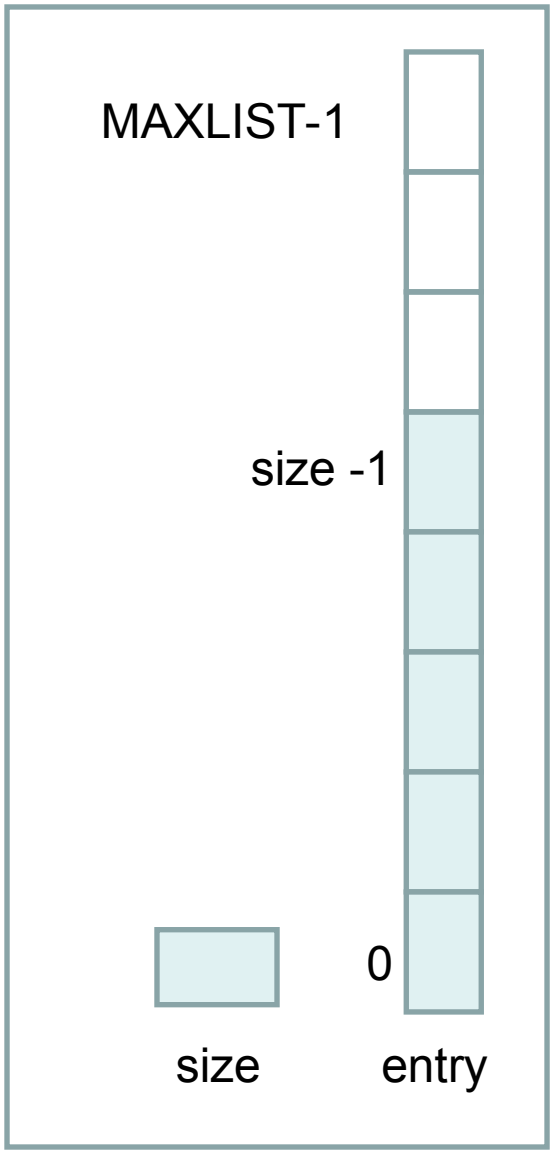
```
void CreateList(List *pl){
      pl->size=0;
} // Θ(1)

int ListEmpty(List *pl){
      return !pl->size;
} // Θ(1)

int ListFull(List *pl){
      return pl->size==MAXLIST;
} // Θ(1)

int ListSize(List *pl){
      return pl->size;
} // Θ(1)

void DestroyList(List *pl){
      pl->size=0;
} // Θ(1)
```



© Waleed A. Yousef 2008

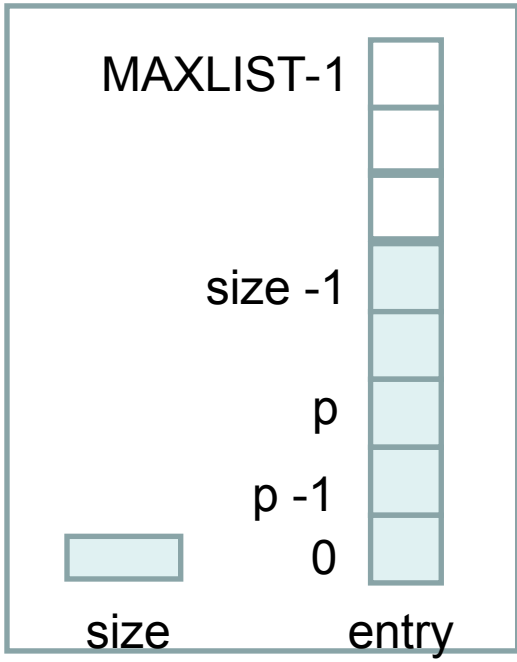
```
/*0 <= p <= size*/
void InsertList(int p, ListEntry e, List *pl){
     int i;
     /*The loop shifts up all the elements in
     the range [p, size-1] to free the pth
location*/
     for(i=pl->size-1; i>=p; i--)
          pl->entry[i+1]=pl->entry[i];
     pl->entry[p]=e;
     pl->size++;
} // Θ(n)
```

Special Cases are all the combination of the following:

$p = 0$ or $p = size$

$size = 0$

All the cases will work



Inserting one element requires too many shifting!!

© Waleed A. Yousef 2008

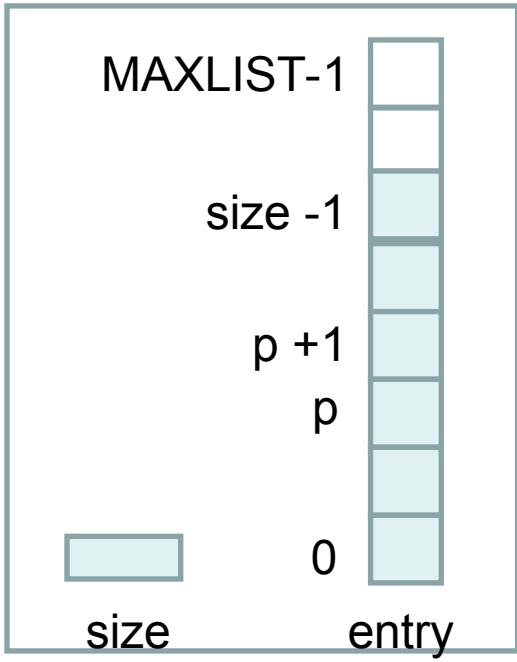
```
/*0<= p <= size-1 and List not empty*/
void DeleteList(int p, ListEntry *pe, List *pl){
     int i;
     *pe=pl->entry[p];
     /*The loop shifts down all the elements in
     the range [p+1, size-1] to free the pth
location*/
     for(i=p+1; i<=pl->size-1; i++)
          pl->entry[i-1]=pl->entry[i];
     pl->size--;
} // Θ(n)
```

Special Cases are all the combination of the following:

$p = 0$ or $p = size - 1$

$size = 1$

All the cases will work

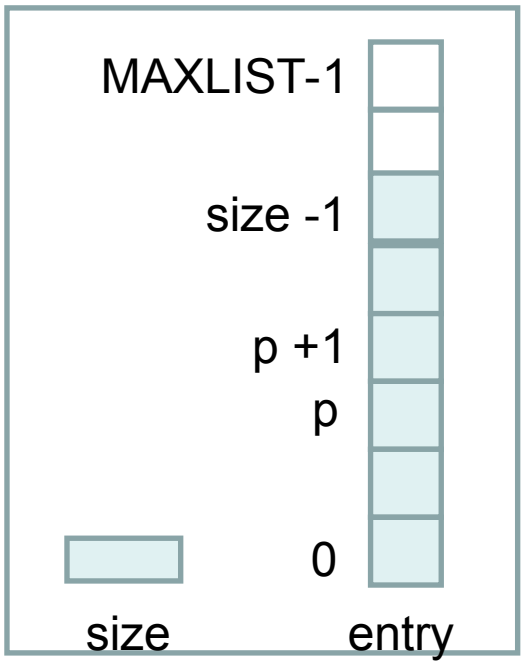


Deleting one element requires too many shifting!!

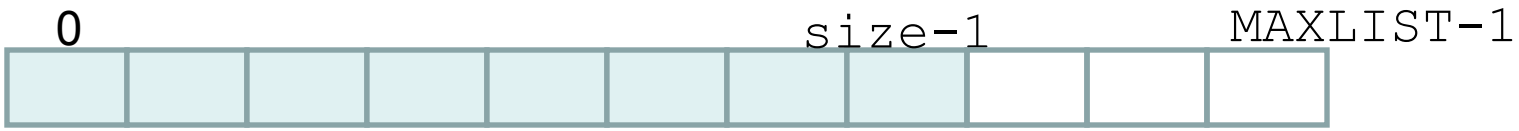
© Waleed A. Yousef 2008

```
/* 0<= p <= size-1*/
void RetrieveList(int p, ListEntry *pe, List *pl){
      *pe=pl->entry[p];
} // Θ(1)
/* 0<= p <= size-1*/
void ReplaceList(int p, ListEntry e, List *pl){
      pl->entry[p]=e;
} // Θ(1)

void TraverseList(List* pl, void (*Visit)(ListEntry)){
      int i;
      for(i=0; i<pl->size; i++)
            (*Visit)(pl->entry[i]);
} // Θ(n)
```



© Waleed A. Yousef 2008



Issues at the user level:

How to insert at the beginning of the List?

```
InsertList(0, e, &l);
```

How to insert at the end of the List?

```
InsertList(ListSize(&l), e, &l);
```

How to use it as a stack?
How to use it as a queue?
Think at home

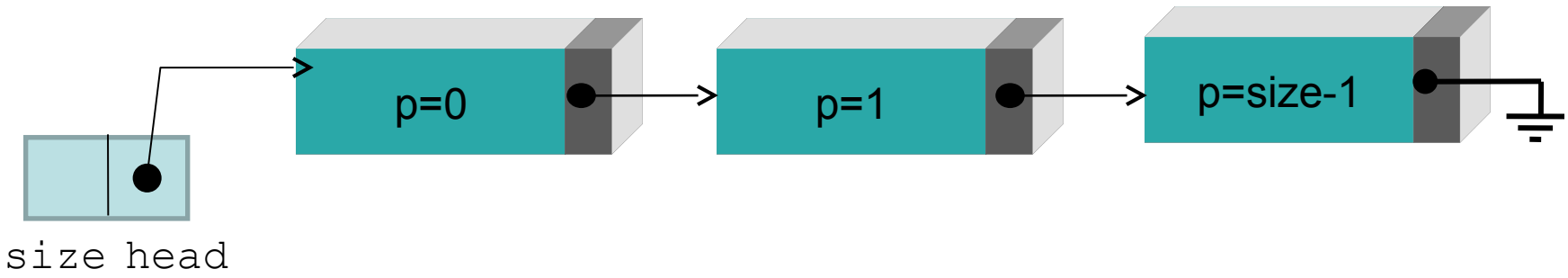
Now let us start the linked Implementation

© Waleed A. Yousef 2008

```c
/*List.h*/
#include "Global.h"

typedef struct listnode{
      ListEntry entry;
      struct listnode *next;
}ListNode;

typedef struct list{
      ListNode    *head;
      int         size;
}List;
```



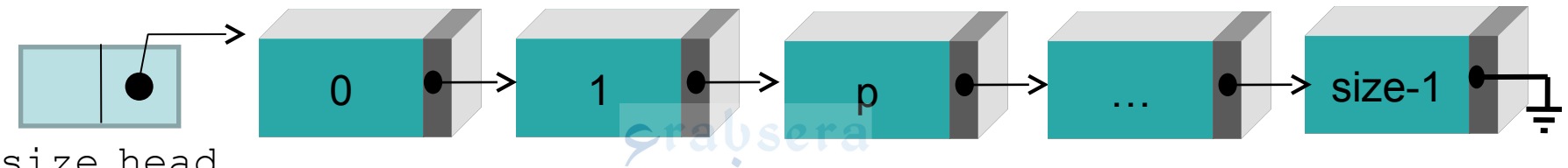
© Waleed A. Yousef 2008

```
void CreateList(List *pl){
      pl->head=NULL;
      pl->size=0;
} // Θ(1)

int ListEmpty(List *pl){
      return (pl->size==0);
      //or return !pl->head
} // Θ(1)

int ListFull(List *pl){
      return 0;
} // Θ(1)

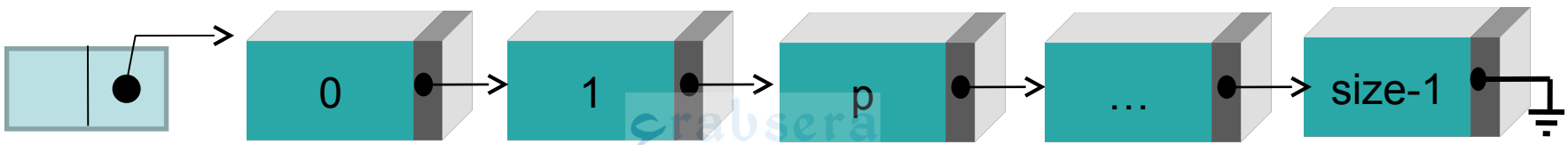
int ListSize(List *pl){
      return pl->size;
} // Θ(1)
```



© Waleed A. Yousef 2008

```
void DestroyList(List *pl){
      ListNode *q;
      while(pl->head){
            q=pl->head->next;
            free(pl->head);
            pl->head=q;
      }
      pl->size=0;
}//we took it before many times: // Θ(n)

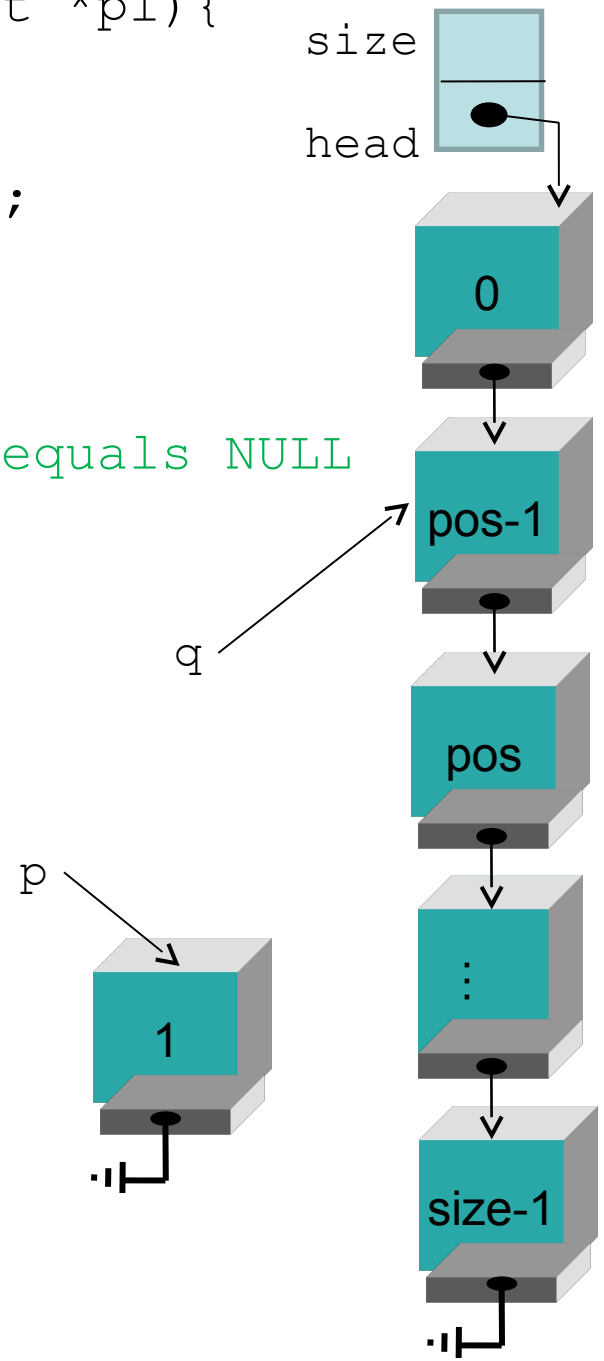
void TraverseList(List* pl, void(*Visit)(ListEntry)){
      ListNode *p=pl->head;
      while(p){
            (*Visit)(p->entry);
            p=p->next;
      }
} // Θ(n)
```



© Waleed A. Yousef 2008

```
void InsertList(int pos, ListEntry e, List *pl){
    ListNode *p, *q;
    int i;
    p=(ListNode *)malloc(sizeof(ListNode));
    p->entry=e;
    p->next=NULL;

    if (pos==0){//will work also for head equals NULL
        p->next=pl->head;
        pl->head=p;
    }
    else{
        for(q=pl->head, i=0; i<pos-1; i++)
                q=q->next;
        p->next=q->next;
        q->next=p;
    }
    pl->size++;
} // Θ(n) but without shifting elements.
```



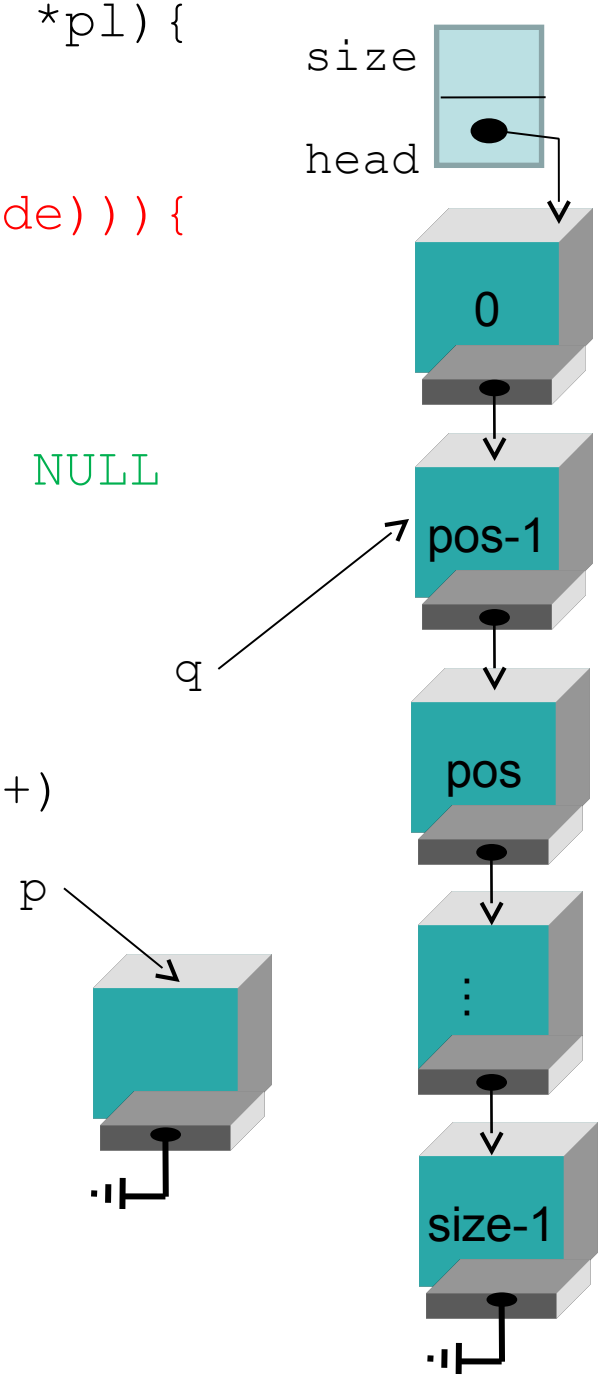
© Waleed A. Yousef 2008

We should make sure that the memory is not full when we call `malloc` (as we did previously). We have to design the function to return `int` not void and we modify the contiguous implementation accordingly to `return 1` always.

© Waleed A. Yousef 2008

```
int InsertList(int pos, ListEntry e, List *pl){
    ListNode *p, *q;
    int i;
    if (p=(ListNode *)malloc(sizeof(ListNode))){
        p->entry=e;
        p->next=NULL;

        if (pos==0){//works also for head = NULL
            p->next=pl->head;
            pl->head=p;
        }
        else{
            for(q=pl->head, i=0; i<pos-1; i++)
                q=q->next;
            p->next=q->next;
            q->next=p;
        }
        pl->size++;
        return 1;
    }
    else return 0;
}
```

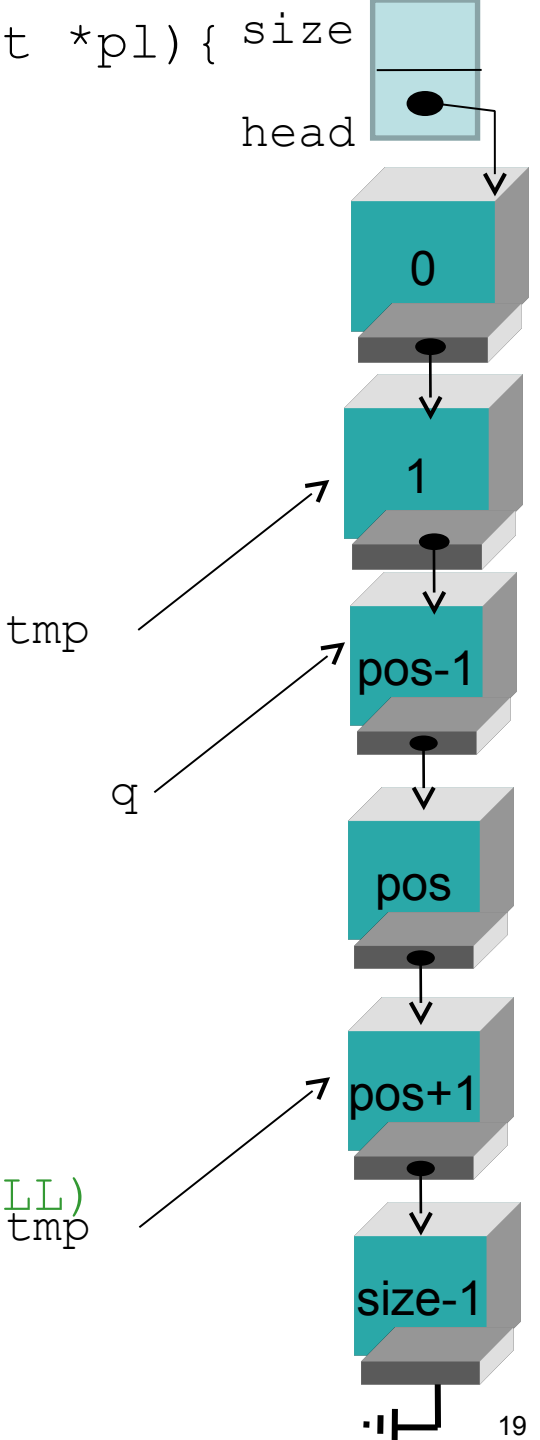


© Waleed A. Yousef 2008

```
void DeleteList(int pos, ListEntry *pe, List *pl){
    int i;
    ListNode *q, *tmp;

    if (pos==0){
        *pe=pl->head->entry;
        tmp=pl->head->next;
        free(pl->head);
        pl->head=tmp;
    }// it works also for one node
    else{
        for(q=pl->head, i=0; i<pos-1; i++)
                q=q->next;

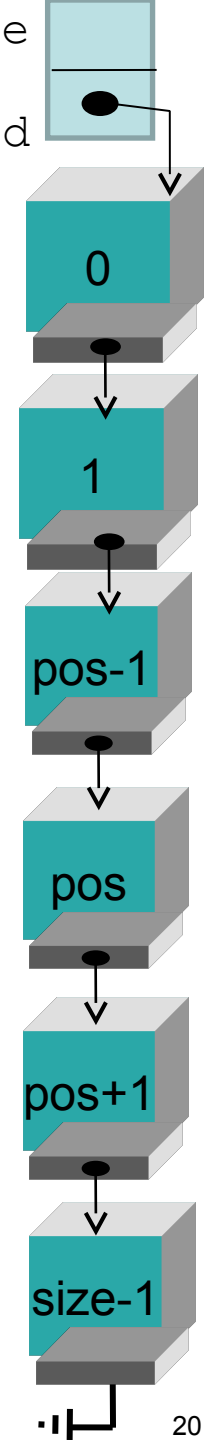
        *pe=q->next->entry;
        tmp=q->next->next;
        free(q->next);
        q->next=tmp;
    }// check for pos=size-1 (tmp will be NULL)
    pl->size--;
} //O(n) but without shifting elements.
```



© Waleed A. Yousef 2008

```
void RetrieveList(int pos, ListEntry *pe, List *pl){
      int i;
      ListNode *q;
      for(q=pl->head, i=0; i<pos; i++)
            q=q->next;
      *pe=q->entry;
}
```

```
void ReplaceList(int pos, ListEntry e, List *pl){
      int i;
      ListNode *q;
      for(q=pl->head, i=0; i<pos; i++)
            q=q->next;
      q->entry=e;
}
```



© Waleed A. Yousef 2008

Comparison between the array-based and the linked implementation: "Which is **always** better?" is a wrong question!

	Array-based	**Linked**
`CreateList`	$\Theta(1)$	$\Theta(1)$
`InsertList`	$\Theta(n)$	$\Theta(n)$
`DeleteList`	$\Theta(n)$	$\Theta(n)$
`RetrieveList`	$\Theta(1)$	$\Theta(n)$
`ReplaceList`	$\Theta(1)$	$\Theta(n)$

InsertList is very time consuming for Array-based because of copying elements, especially if the elements are large records.

RetrieveList and ReplaceList are always better for contiguous implementation.

Read the book very well.

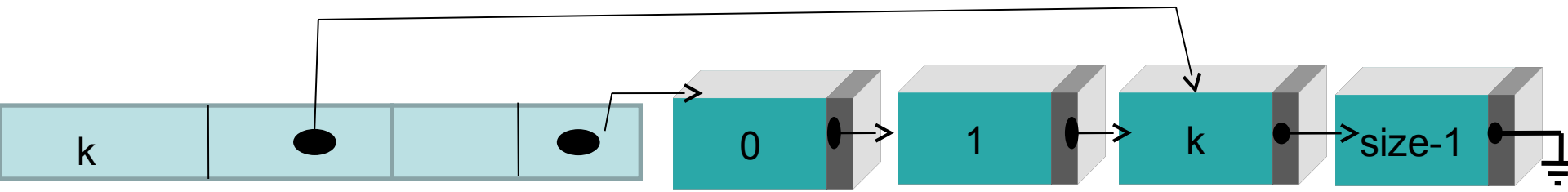
© Waleed A. Yousef 2008

Design Enhancement: Learn how you modify your design to enhance the performance

Many applications processes the entries in order, i.e., moving from one entry to the next.

Many other applications refer to the same entry many times.

Then, our current linked implementation is very inefficient, since it moves from the head to the element every time!
Then, we need to *remember* the last position `currentpos` and start navigating from it, and we use `current` to start walking from `currentpos`



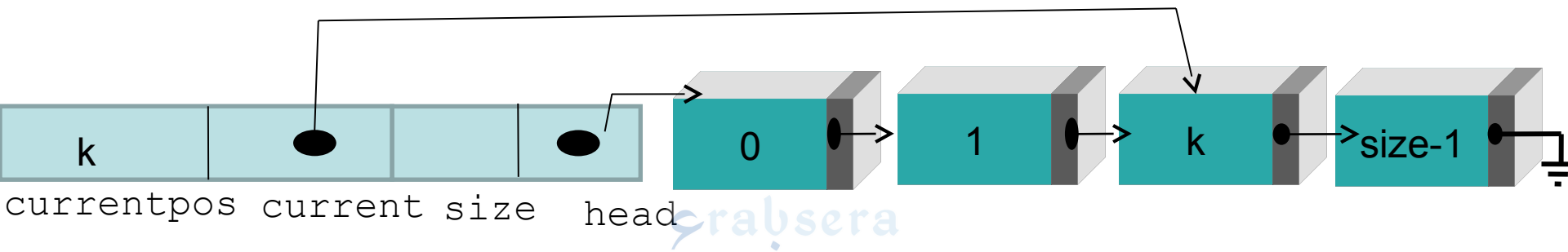
© Waleed A. Yousef 2008

Design Enhancement: Learn how you modify your design to enhance the performance

Of course, this will not help if the new element is preceding the last element visited.

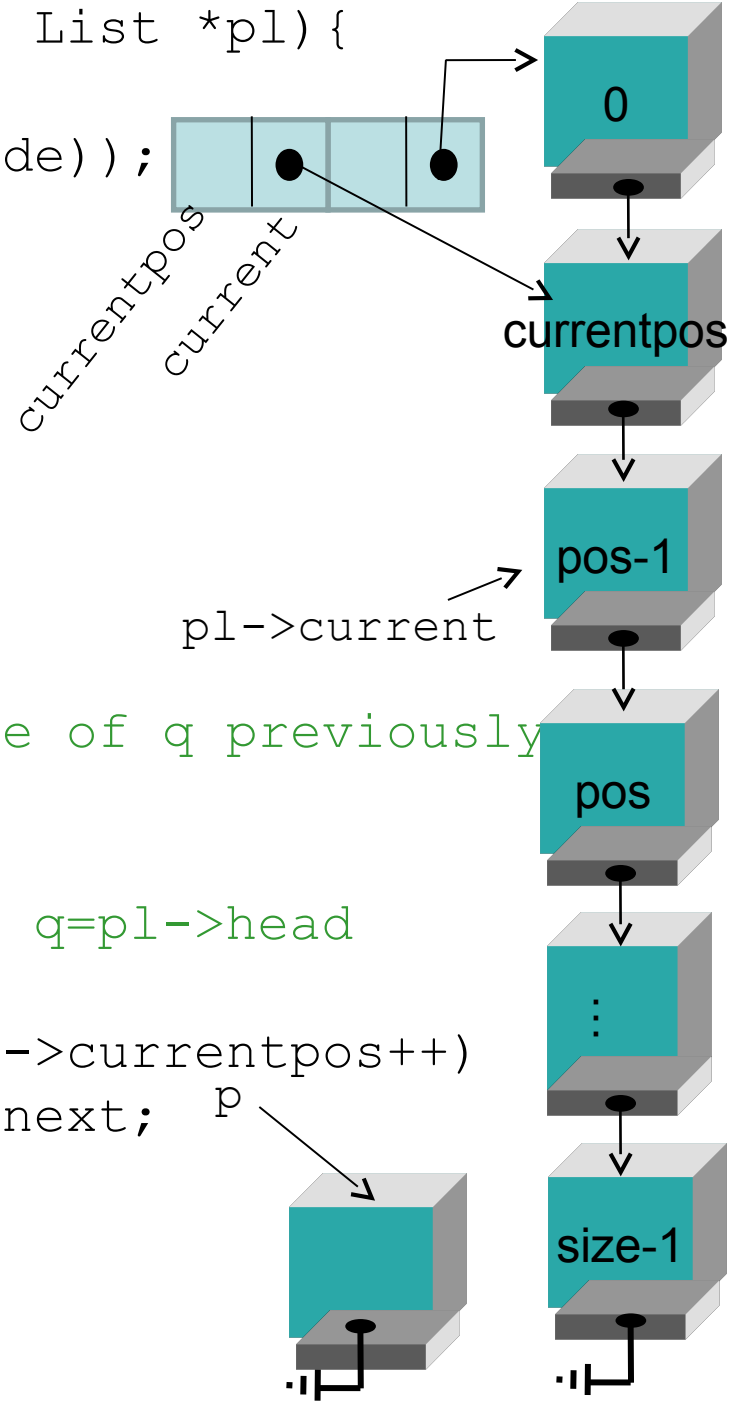
Only the type definition, `InsertList`, `DeleteList`, `ReplaceList`, and `RetrieveList` will change.

```
typedef struct list{
        ListNode *head, *current;
        int       size, currentpos;
}List;
```



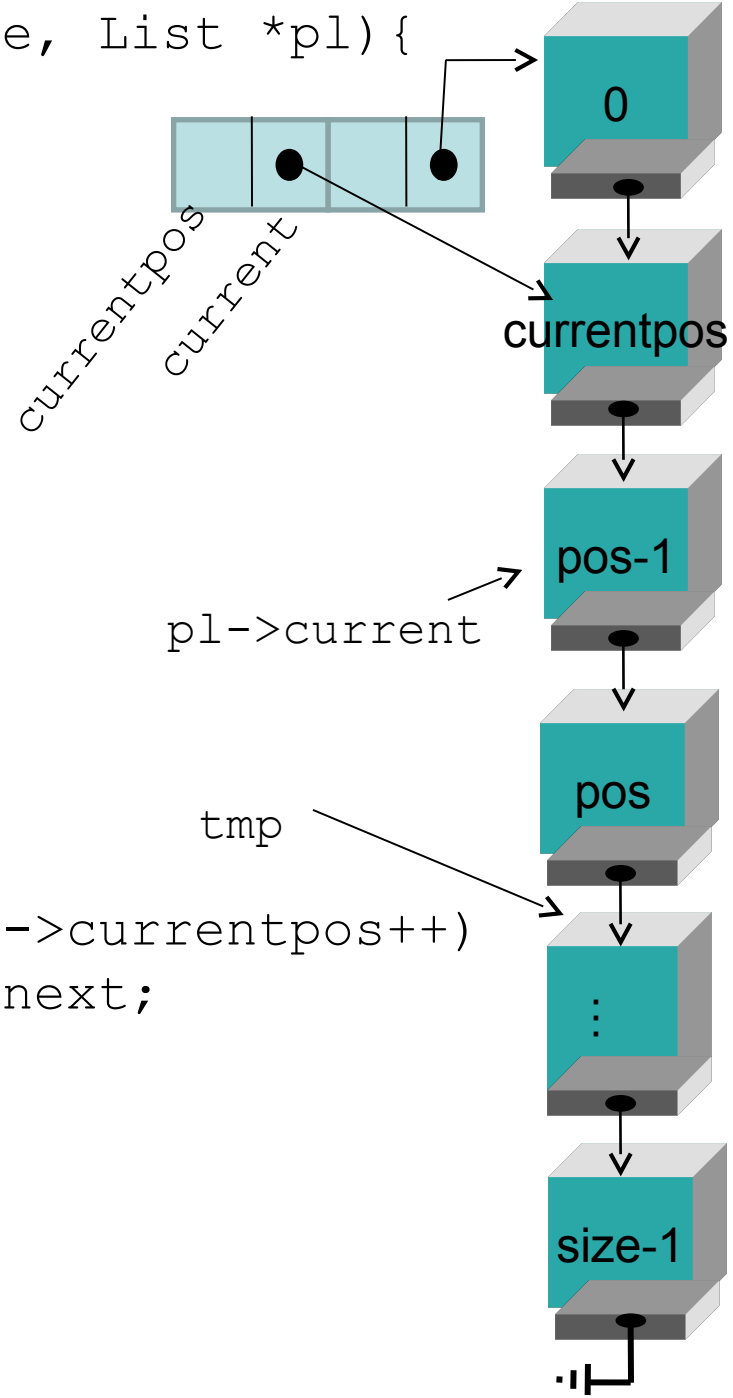
© Waleed A. Yousef 2008

```
void InsertList(int pos, ListEntry e, List *pl){
    ListNode *p;
    p=(ListNode *)malloc(sizeof(ListNode));
    p->entry=e;
    p->next=NULL;
    if (pos==0){
        p->next=pl->head;
        pl->head=p;
        pl->currentpos=0; //new
        pl->current=pl->head; //new
    }
    else{//pl->current is used in place of q previously
        if (pos<=pl->currentpos){
            pl->currentpos=0;//as i=0
            pl->current=pl->head;//as q=pl->head
        } //new.
        for(; pl->currentpos!=pos-1; pl->currentpos++)
            pl->current=pl->current->next;
        p->next=pl->current->next;
        pl->current->next=p;
    }
    pl->size++;
```



©\Waleed A. Yousef 2008

```
void DeleteList(int pos, ListEntry *pe, List *pl){
    ListNode *tmp;
    if (pos==0){
        *pe=pl->head->entry;
        pl->current=pl->head->next;
        free(pl->head);
        pl->head=pl->current; //new
        pl->currentpos=0; //new
    }
    else{
        if (pos<=pl->currentpos){
                pl->currentpos=0;
                pl->current=pl->head;
        }
        for(; pl->currentpos!=pos-1; pl->currentpos++)
                pl->current=pl->current->next;
        *pe=pl->current->next->entry;
        tmp=pl->current->next->next;
        free(pl->current->next);
        pl->current->next=tmp;
    }
        pl->size--;
```



©Waleed A. Yousef 2008

You have to modify `ReplaceList` and `RetrieveList` in the same manner. (Do it as a homework). Check also for other functions, e.g., `CreateList` (for initialization)

Compare the previous functions to the functions of the book, where it has a function `SetPosition` that is called from within `InsertList` and `DeleteList`.

Having this function may simplify the code for the case of having `current` and `currentpos`.

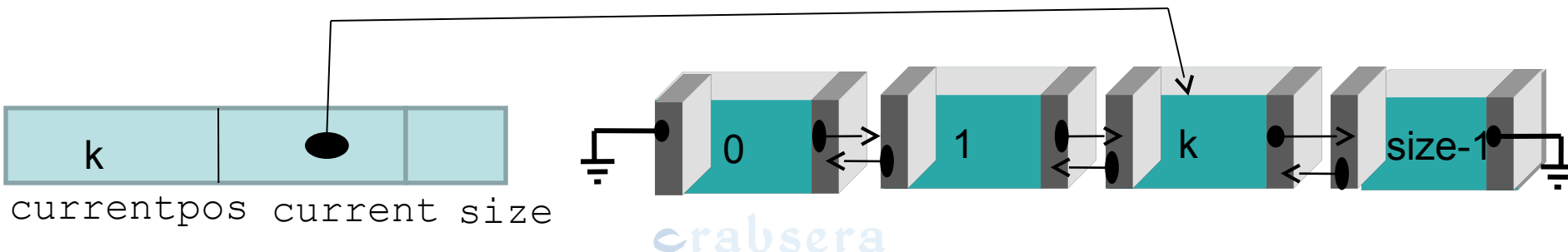
© Waleed A. Yousef 2008

Design Enhancement: Learn how you modify your design to enhance the performance

Accessing the list at a position preceding `currentpos` will be slow, since we cannot move back. A possible remedy is using doubly linked list.

We need just a pointer, `current`, not necessarily point to the first node. `currentpos` will always indicates the order of the current node.

Read the code from the book and solve the review problems



© Waleed A. Yousef 2008